eg89.cpp

```
#include<iostream>
using namespace std;
inline void do_something()
{
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
```

```
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
}
int main()
{
do_something();
cout<<"Ujjain"<<endl;
do_something();
cout<<"Ujjain"<<endl;
do_something();
return 0;
}
```

Thinking Machines Classroom Codes

eg90.cpp

```
#include<iostream>
using namespace std;
__attribute__((always_inline))
inline void do_something()
{
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
```

```
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
cout<<"God is great"<<endl;
}
int main()
{
do_something();
cout<<"Ujjain"<<endl;
do_something();
cout<<"Ujjain"<<endl;
do_something();
return 0;
}
```

Thinking Machines Classroom Codes

eg91.cpp

```
#include<iostream>
using namespace std;
void just_do_it()
{
alignas(64) int x;
alignas(64) int y;
cout<<(uintptr_t)&x<<endl;
cout<<(uintptr_t)&y<<endl;
}
int main()
{
int x;
int y;
cout<<(uintptr_t)&x<<endl;
cout<<(uintptr_t)&y<<endl;
just_do_it();
return 0;
}
```

Thinking Machines Classroom Codes

eg92.cpp

```
// to be completed
#include<iostream>
using namespace std;
class Bulb
{
private:
int wattage;
int price;
public:
explicit Bulb()
{
this->wattage=0;
this->price=10;
}
explicit Bulb(int wattage,int price):wattage(wattage),price(price){}
void set_wattage(int wattage)
{
this->wattage=wattage;
}
int get_wattage()
{
return this->wattage;
}
int set_price(int price)
{
this->price=price;
}
int get_price()
{
return this->price;
}
};
template<typename T>
class memory_pool
{
private:
union node
{
 node();  ~node();
      T obj;
 node *next;
};
node *list;
size_t sz;
node *head;
public:
memory_pool(size_t sz)
{
```

```
node *head;
this->sz=sz;
this->list=new node[sz];
head=nullptr;
// setup linked list
this->head=head;
}
}
void * acquire()
{
}
void release(void *p)
{
}
bool is_available()
{
}
};
int main()
{
memory_pool<Bulb>  pool(5);
Bulb *b1=nullptr;
Bulb *b2=nullptr;
b1=new (pool.acquire()) Bulb;
if(b1)
{
}
b2=new (pool.acquire()) Bulb(60,10);
if(b2)
{
}
pool.release(b1);
pool.release(b2);
return 0;
}
```

eg93.cpp

```
#include<iostream>
#include<utility>
using namespace std;
void do_something(string g,int x)
{
cout<<g<<endl;
cout<<x<<endl;
}
void do_something_else(int e,int f,string m)
{
cout<<e<<endl;
cout<<f<<endl;
cout<<m<<endl;
}
template<typename T,typename... Args>
void just_do_it(T k,Args... args)
{
k(forward<Args>(args)...);
}
int main()
{
just_do_it(do_something,"golu",100);
just_do_it(do_something_else,10,20,"golu");
return 0;
}
```

eg94.cpp

```
#include<iostream>
#include<utility>
using namespace std;
void do_something(string g,int x)
{
cout<<g<<endl;
cout<<x<<endl;
}
void do_something_else(int e,int f,string m)
{
cout<<e<<endl;
cout<<f<<endl;
cout<<m<<endl;
}
class Whatever
{
public:
template<typename T,typename... Args>
void just_do_it(T k,Args... args)
{
k(forward<Args>(args)...);
}
};
int main()
{
Whatever w;
w.just_do_it(do_something,"golu",100);
w.just_do_it(do_something_else,10,20,"golu");
return 0;
}
```

eg95.cpp

```
#include<iostream>
using namespace std;
class aaa
{
public:
void operator()()
{
cout<<"great"<<endl;
}
void operator()(int e,int f)
{
cout<<e+f<<endl;
}
};
int main()
{
aaa kalia;
kalia(10,20);  // this is possible because, kalia is a callable object
kalia(); // this is possible because, kalia is a callable object
return 0;
}
```

eg96.cpp

```
#include<iostream>
using namespace std;
char * get_it()
{
return new char;
}
int main()
{
char *p=get_it();
char *j;
j=new (p) char;
cout<<(uintptr_t)p<<endl;
cout<<(uintptr_t)j<<endl;
char *i=new (get_it()) char;
cout<<(uintptr_t)i<<endl;
delete i;
delete p;
return 0;
}
```

eg97.cpp

```
#include<iostream>
using namespace std;
int main()
{
char *p;
p=new (nullptr) char;
cout<<(p==nullptr)<<endl;
return 0;
}
```

eg98.cpp

```
#include<iostream>
#include<cstdlib>
using namespace std;
void * get_it()
{
return malloc(1);
}
int main()
{
char *p;
p=new (get_it()) char;
cout<<(uintptr_t)p<<endl;
free(p);
return 0;
}
```

eg99.cpp

```
int main()
{
int x;
int *p;
p=&x;
void *k;
k=p;
return 0;
}
```

Thinking Machines Classroom Codes